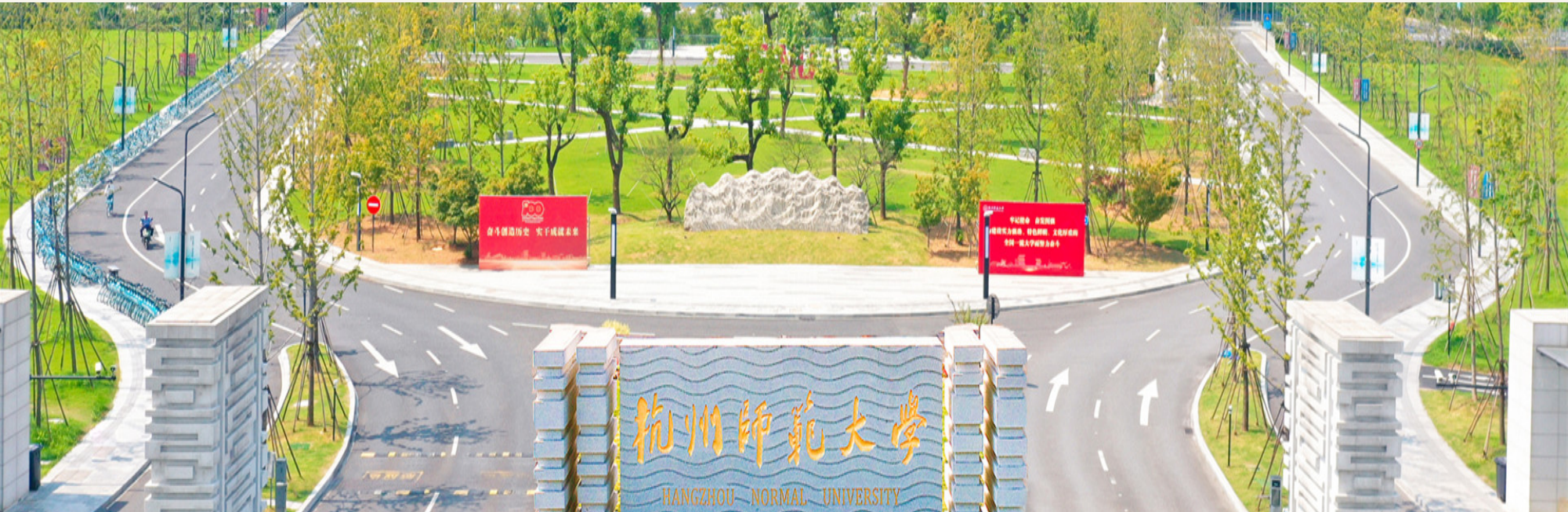
第二讲 - NLP基础算法：正则表达式与文本标准化

张建章

阿里巴巴商学院
杭州师范大学

2026-03-01



1 正则表达式基础

2 语料库

3 文本预处理

4 分词

5 最小编辑距离

6 课后实践

正则表达式的定义与作用

正则表达式（Regular Expression，简称regex）是一种用于匹配和操作字符串的语言。在文本处理中，正则表达式广泛应用于文本模式匹配，可以帮助查找、替换、拆分字符串。其核心任务是定义一组规则来描述需要匹配的文本模式，通过这些模式，能够有效地在文本中进行搜索、替换、提取等操作。

正则表达式基本操作

正则表达式通过一系列的字符和符号组成，执行不同的匹配操作。其最基础的操作为字面字符匹配。

字面字符匹配：将字符按照顺序组合 (Concatenation)来进行匹配。可以简单地使用一个正则表达式来查找一段固定的字符序列。

例如，查找文本中是否有“市场营销”这一具体词组：

```python
import re

text = "我们公司最近进行了市场营销活动，市场营销效果非常好。"
# 定义正则表达式
pattern = r"市场营销"
# 使用re.search来查找匹配
result = re.search(pattern, text)
if result:
  print(f"匹配成功: {result}")
else:
  print("没有匹配到")
```

例如，查找文本中是否有“销售”和“广告”这两个词语。

```
text = "我们的销售和广告团队合作密切，取得了显著成绩。"

# 定义正则表达式
pattern = r"销售|广告"

# 使用re.findall来查找所有匹配
matches = re.findall(pattern, text)

print(f"匹配到的词语: {matches}")
```

正则表达式 `r"销售|广告"` 使用 | 表示“或”操作，匹配文本中的“销售”或“广告”字样。 `re.findall()` 会返回所有匹配的结果。

`re.match(pattern, string)`：从字符串的最开始位置进行匹配。如果字符串的第一个字符就不符合规则，它会立刻放弃，返回 `None` 。

`re.search(pattern, string)`：扫描整个字符串，直到找到第一个匹配的子串为止。如果扫描完整个字符串都找不到，才返回 `None` 。

字符集与字符范围

正则表达式提供了字符集和字符范围的功能，使得用户能够灵活地匹配多种字符，字符集和字符范围允许匹配一组字符中的任意一个字符：

- **字符集**：使用方括号 [] 定义字符集，匹配集合中的任意一个字符。例如，[wW] 会匹配 w 或 W；
- **字符范围**：使用连字符 - 定义字符的范围。例如，[a-z] 匹配任意一个小写字母，[0-9] 匹配任意一个数字；
- **否定字符集**：使用 ^ 作为字符集的第一个字符，表示匹配集合中不包含的字符。例如，[^a] 匹配任何非 a 字符。

Regex	Expansion	Match	First Matches
\d	[0-9]	any digit	Party␣of␣5
\D	[^0-9]	any non-digit	Blue␣moon
\w	[a-zA-Z0-9_]	any alphanumeric/underscore	Daiyu
\W	[^\w]	a non-alphanumeric	!!!!
\s	[␣\r\t\n\f]	whitespace (space, tab)	in_Concord
\S	[^\s]	Non-whitespace	in␣Concord

图 1: 常用字符集别名

例如，查找文本中所有以“广”或“销”开头的词语。

```
text = "我们正在进行广泛的市场推广和销售活动。"

# 定义正则表达式
pattern = r"[广销]\w+?"
# 使用re.findall来查找所有匹配
matches = re.findall(pattern, text)
print(f"匹配到的词语: {matches}")
# 匹配到的词语: ['广泛', '广和', '销售']
```

① ?表示非贪婪模式，匹配满足条件的最短字符串；② Python 3的re模块默认启用了Unicode支持，\w被定义为匹配Unicode“字符”，包括中文字符，不包括标点符号；

注意：基本汉字字符集的正则表达式为 `r'[\u4e00-\u9fff]'` 。

练习：从一段中英文混合的文本中提取出所有的汉字内容。

量词

指定一个字符或子表达式出现的次数。常见的量词有 *、+、? 和 {n,m}：

- *：匹配前一个字符零次或多次（例如，a* 会匹配零个或多个 a）；
- +：匹配前一个字符一次或多次（例如，a+ 会匹配一个或多个 a）；
- ?：匹配前一个字符零次或一次（例如，colou?r 会匹配 color 或 colour）；
- {n}：匹配前一个字符恰好出现 n 次（如，[0-9]{3} 匹配恰好三个数字）；
- {n, m}：匹配前一个字符出现n次（最少）到m次（最多）。

Regex	Match
*	zero or more occurrences of the previous char or expression
+	one or more occurrences of the previous char or expression
?	zero or one occurrence of the previous char or expression
{n}	exactly *n* occurrences of the previous char or expression
{n,m}	from *n* to *m* occurrences of the previous char or expression
{n,}	at least *n* occurrences of the previous char or expression
{,m}	up to *m* occurrences of the previous char or expression

图 2: 常用量词

例如，查找文本中所有“...的...”形式的短语。

```
text = "这个产品的市场推广的效果非常好，值得关注的点很多。"

# 定义正则表达式
pattern = r"\w{,2}的\w{,2}"

# 使用re.findall来查找所有匹配
matches = re.findall(pattern, text)

print(f"匹配到的词语: {matches}")
# ['产品的市场', '推广的效果', '关注的点很']
```

正则表达式 `r"\w{,2}的\w{,2}"` 查找文本中所有符合“最多两个字符+的+最多两个字符”模式的子字符串。

特殊字符

正则表达式中有一系列特殊字符，这些字符具有特定的功能：

- .（点号）：匹配任意单个字符（除了换行符）；
- ^（脱字符）：匹配字符串的开始位置；
- $（美元符号）：匹配字符串的结束位置；
- \b：匹配单词字符边界，字符与非字符之间的位置；
- \B：匹配非单词边界，字符与字符之间的位置。

```python
import re
text = "这家公司最近扩展了其业务，成效显著。"
pattern = r"这.公司"
result = re.search(pattern, text)
print(result.group())  # 输出: "这家公司"
```

```python
pattern = r"^这家公司"
result = re.match(pattern, text)
print(result.group())  # 输出: "这家公司"
```

```python
pattern = r"显著。$"
result = re.search(pattern, text)
print(result.group())  # 输出: "显著。"
```

\b匹配单词边界字符，即匹配单词和非单词字符之间的位置（如字符与空格、标点符号之间的位置）。“务”字与逗号之间就是一个单词边界（看不见的）。

```
text = "这家公司最近扩展了其业务，其业务成效显著。"
pattern = r"业务\b"
result = re.findall(pattern, text)
print(result)  # 输出: ['业务']
```

\B匹配非单词边界字符，即匹配在单词字符与单词字符之间的位置。“著”字与句号之间是一个单词边界，“效”字与“显”字之间是一个非单词边界（看不见的）

```
pattern = r"显著\B"
result = re.findall(pattern, text)
print(result)  # 输出: []
```

分组与捕获组

正则表达式可以使用圆括号 () 来将表达式进行分组。分组不仅可以改变运算符的优先级，还可以为分组的内容创建捕获组，以便后续引用。

- **分组**：使用圆括号 () 可以将多个字符“打包”成一个整体，以便对这个“整体”统一应用量词（如重复几次）或逻辑（如或运算）。例如，(abc){2}匹配字符串 abc两次；
- **捕获组**：圆括号不仅能“打包”，还会像收纳盒一样，把括号内匹配到的具体内容“暂存”起来并自动编号。

```
text = "公司A的季度财报显示利润增长30%，表现出色。
↪  公司B的销售额增长了15%，也表现良好。"
pattern = r"公司([A-Z])的([\u4e00-\u9fff]+)"  #
↪  匹配公司名和其业绩描述
result = re.findall(pattern, text)
# [('A', '季度财报显示利润增长'), ('B', '销售额增长了')]
r = re.search(pattern, text)
r.group(1), r.group(2), r.group(0) # ('A', '季度财报显示利润增长',
↪  '公司A的季度财报显示利润增长')
```

应用场景与实现

正则表达式在各种文本处理任务中都有广泛应用，常见场景包括：

- **文本搜索**：查找文件或字符串中是否含有特定模式；
- **替换操作**：使用正则表达式进行文本替换，例如删除特定模式的文本。
- **数据验证**：验证文本是否符合特定格式，例如电子邮件、电话号码等。

正则表达式在不同的工具和编程语言中有不同的实现方式。在 Unix 系统中，grep 命令就是基于正则表达式进行文本搜索的工具。在 Python 中，re 模块提供了正则表达式的实现，允许用户通过编程实现匹配、搜索和替换等操作。

练习：利用正则表达式替换各种形式的中括号引用，例如：

`[cite: 2, 3]`或`[cite_start]`或`[cite: 11]`

大模型依据用户文件给出的回复的源代码中经常出现符合上面模式的文本片段，表示对用户文件内容的引用，复制大模型回复的源代码使用时，必须删除。大模型回复内容的源代码通常为markdown。

语料库的特点

语料库（corpus）是进行自然语言处理（NLP）的基础，它指的是一组有代表性的、经过精心收集和标注的文本或多模态数据集。在NLP研究中，语料库不仅是数据分析的基础，更能反映出语言使用的特定背景、语境和社会特征。因此，语料库的构建与选择对NLP任务的有效性至关重要。

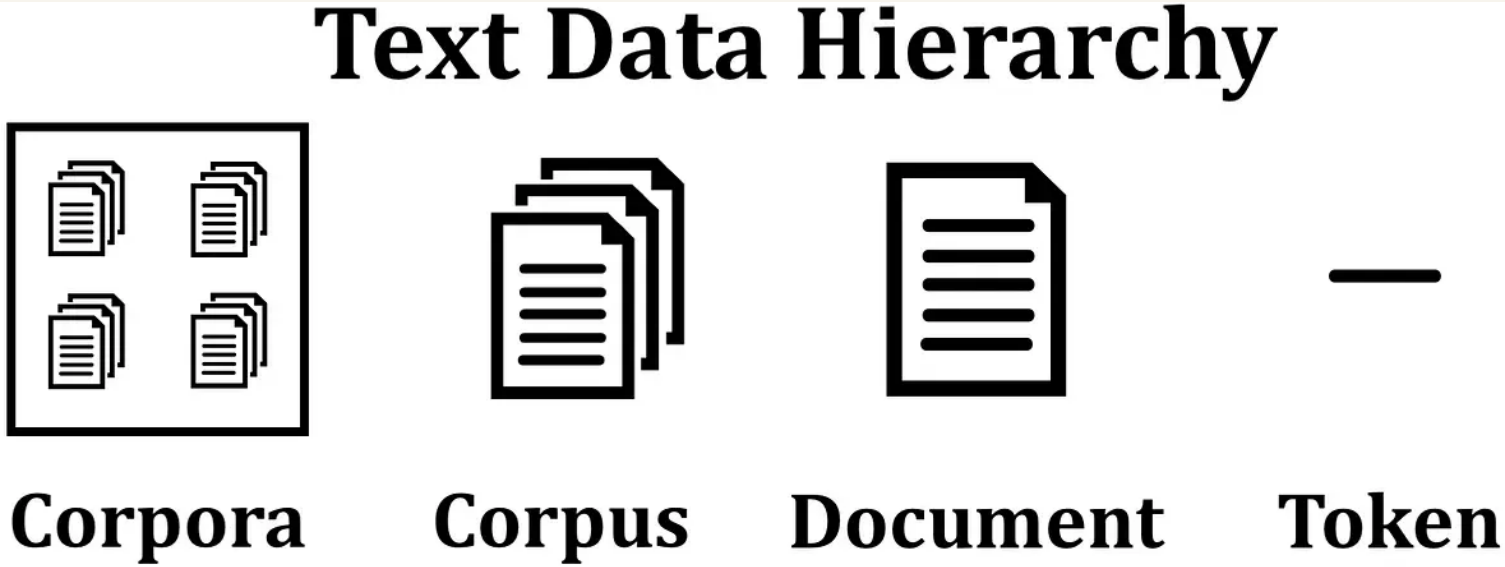


图 3: 语料库的层次结构

- **语言多样性**：世界上共有约7097种语言，而NLP算法大多是在英语语料库上开发和测试的。这种情况可能导致算法在处理其他语言时遇到困难，理想的NLP算法应能够处理多种语言。因此，在构建语料库时，研究者需要考虑语言的多样性，确保语料库能够代表不同语言的特点。

- **语料库中的体裁差异**：不同体裁的文本在语言使用上有很大不同。例如，新闻报道、小说、学术论文、社交媒体和法律文件等文本形式都有其特定的表达方式和语言特点。因此，在进行文本分析时，研究者必须选择适合的语料库，并考虑体裁差异的影响。

- **语料库中的社会人口特征**：语料库的语言使用还受到说话者或写作者的社会人口特征的影响。这些特征包括年龄、性别、种族、社会阶层等，它们都会在一定程度上影响语言的表达方式。理解这些社会人口特征对语言的影响，有助于提高语料库分析的准确性和应用的适应性。

- **语料库的历史维度**：语言是动态变化的，随着时间的推移，语言的结构和词汇会发生变化。特别是在处理历史文本或长时间跨度的数据时，理解语言随时间变化的特点非常重要。

- **语料库的应用与数据声明**：为了确保语料库的质量和可信度，语料库创建者应提供详细的数据说明，明确语料库的收集动机、语料来源、收集过程、数据预处理、注释过程以及数据分发方式等信息。

句子预处理：句子分割是自然语言处理中不可忽视的一步，其准确性直接影响到后续文本处理任务的效果。由于标点符号具有多重含义，句子分割不仅依赖标点符号，还需要结合上下文、词性标注以及其他语言特征来判断句子的边界。随着机器学习方法的引入，句子分割的精度有了显著提升，但仍需要根据特定任务和语言特点进行调整和优化。

词汇预处理：分词、词性标注、词汇标准化、词形还原和词干提取是文本预处理中的关键步骤，它们通过不同的方式简化和统一文本中的词语形式。

- 词汇标准化通常涉及大小写统一和同义词归一化；
- 词形还原通过分析单词的词缀和词根来还原单词的基本形式；
- 词干提取则是通过简单的规则去掉词缀来提取词干。

Tokenization的两种方式

分词（Tokenization）是文本处理中的基础任务之一，指的是将连续的文本划分为一个个有意义的单位（即token），这些单位通常为单词、子词或字符。文本分词的两种主要方法：自顶向下分词和子词分词。

自顶向下分词：依赖规则或预定义的词典，适合处理结构清晰的文本；

子词分词：将词语进一步拆解为更小的单元（子词、字符或字母）来处理文本，能够有效处理未登录词和复杂语言结构。子词分词方法，如BPE，能够为现代NLP模型提供更加灵活且强大的文本处理能力，尤其是在处理大规模语料和多语言任务时具有显著优势。

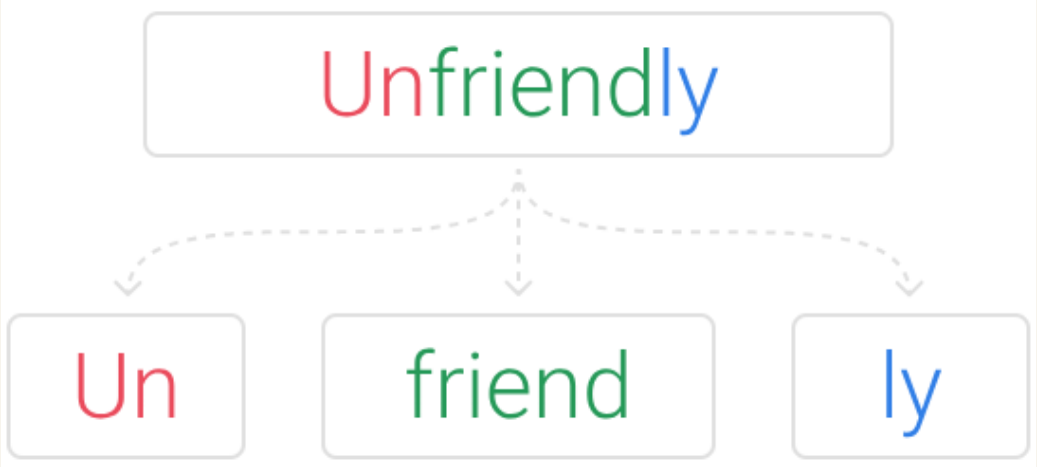


图 4: 子词 (subword)分词

Byte-Pair Encoding (BPE)方法

BPE是一种基于统计的自底向上的分词方法，广泛应用于现代大规模语言模型的训练中。BPE的基本思想是通过合并文本中最常见的字符对，逐步构建出新的子词单元。具体过程如下：

- 从单字符开始，将所有文本中的字符视为独立的token (token learner)；
- 统计文本中最常见的字符对，并将它们合并成新的token，直到合并达到预定次数为止 (token learner)；
- 合并后，使用这些新的子词单元进行文本的分词 (token segmenter)。

分词时，先把文本切分为单子符，然后按照学习到的合并顺序 (即，token被学习到的顺序)贪婪地执行合并操作。

BPE的优势在于能够处理未登录词（OOV，Out-of-Vocabulary），即在训练数据中没有出现过的词。通过将未登录词拆分为更小的已知子词单元，BPE能够有效地处理这些词汇，并且减少词汇表的大小。

function BYTE-PAIR ENCODING(strings C, number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens k times
$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
$V \leftarrow V + t_{NEW}$ # update the vocabulary
Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Figure 2.13 The token learner part of the BPE algorithm for taking a corpus broken up into individual characters or bytes, and learning a vocabulary by iteratively merging tokens. Figure adapted from Bostrom and Durrett (2020).

图 5: BPE子词分词算法 (token learner part)

merge	current vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

图 6: BPE子词分词算法运行示例 (token learner part)

词与子词 (Word V.S. Subword Tokenization)

- **基本概念：从“整词”到“语素”**
  - **词级 (Word)**：以完整单词为基础单元，适用于简单任务。
  - **子词级 (Subword)**：将词汇拆解为更小的语素（前缀、词根、后缀）。例如：*unhappiness* $\rightarrow$ *un* + *happi* + *ness*。
- **核心痛点：为什么引入子词？**
  - **解决未登录词**：通过组合已知子词推测生僻词/新词含义，大幅提升模型鲁棒性。
  - **应对复杂形态语言**：高效处理词形变化丰富（如阿拉伯语）或存在超长复合词（如德语）的语言。
- **行业标杆：大模型的默认基石**
  - 主流 NLP 模型（如 BERT, GPT 系列）均全面采用子词算法（如 BPE, WordPiece），兼顾处理效率与词表大小。
- **特殊场景：无空格语言的处理**
  - 中文、日文等无天然空格分隔，需依赖特定分词算法或基于字符/子词来精准识别边界。

最小编辑距离 (Minimum Edit Distance)

最小编辑距离：衡量两个字符串之间相似度的一个度量标准，它表示通过插入、删除或替换操作，将一个字符串转换为另一个字符串所需的最小操作次数。该指标在商务数据清洗与挖掘中具有重要价值。

- **企业实体对齐 (Entity Resolution)**：在合并不同来源的金融数据库时，量化并识别“Alibaba Group”与“Alibba Grp”是否指代同一家公司；
- **电商搜索拼写纠错 (Spell Correction)**：当用户在平台输入错误的品牌或商品名时，底层系统快速召回编辑距离最近的正确词汇；
- **客户记录去重 (Data Deduplication)**：清洗 CRM（客户关系管理）系统中的脏数据，通过计算相似度合并重复填写的客户信息。

【案例：品牌名模糊匹配】

假设抓取到的脏数据为：`Tencentt`，标准词库中目标词为：`Tencent`

操作：删除末尾多余的字母't'。

结论：最小编辑距离为 1。由于距离极小，可高置信度地将其归一化为`Tencent`。

最小编辑距离的计算

最小编辑距离计算的基本操作包括：

- **插入（Insertion）**：在一个字符串中插入一个字符；
- **删除（Deletion）**：删除一个字符；
- **替换（Substitution）**：将一个字符替换为另一个字符。

Levenshtein（/ˈlɛvənʃtaɪn/）距离：插入、删除和替换的代价都是1。最小编辑距离即为将一个字符串通过这些操作转换为另一个字符串所需的最小代价。

Levenshtein 距离的一个变体是只允许插入和删除操作，替换操作等价于一次删除加一次插入，因此替换操作的代价为2。

动态规划求解最小编辑距离 I

1. 定义状态与初始化

定义一个二维数组$D[i][j]$，表示将字符串X的前i个字符转换为Y的前j个字符的最小编辑距离。i表示X中的字符索引，j表示Y中的字符索引。初始化时：

- $D[0][j]$：表示将空字符串X转换为Y的前j个字符，显然，所需操作数是j次插入操作（插入$Y[j]$中的每一个字符）。

- $D[i][0]$：表示将X的前i个字符转换为空字符串，显然，所需操作数是i次删除操作（删除$X[i]$中的每一个字符）。

2. 递推公式

根据递推公式计算$D[i][j]$的值。递推公式如下：

$$D[i,j] = \min \begin{cases} D[i-1,j]+1, \\ D[i,j-1]+1, \\ D[i-1,j-1] + \begin{cases} 2, & \text{if source}[i] \neq \text{target}[j] \\ 0, & \text{if source}[i] = \text{target}[j] \end{cases} \end{cases}$$

动态规划求解最小编辑距离 II

- $D[i-1,j]+1$ 表示将X的第i个字符删除；
- $D[i,j-1]+1$ 表示在X的末尾插入Y的第j个字符；
- $D[i-1,j-1]+\text{cost}(X[i],Y[j])$ 表示如果X的第i个字符与Y的第j个字符不同，则需要替换。

3. 逐步计算

根据递推公式逐个计算$D[i][j]$的值，填充二维数组。

以Levenshtein 距离变体计算为例，以上述算法过程如下；

动态规划求解最小编辑距离 I

1. 定义状态与初始化

定义二维矩阵 $D[i][j]$：表示将源字符串 X 的前 i 个字符，转换为目标字符串 Y 的前 j 个字符，所需的最小操作次数。

边界初始化（极端情况）：

- $D[0][j]$：X 为空。要把空字符串变成 Y 的前 j 个字，只能全部插入（操作 j 次）。

- $D[i][0]$：Y 为空。要把 X 的前 i 个字变成空，只能全部删除（操作 i 次）。

动态规划求解最小编辑距离 II

2. 递推公式：三种方案的“最优选”

计算 $D[i][j]$ 时，比较三种可能的操作，取代价最小的 (min)：

$$D[i,j] = \min \begin{cases} D[i-1,j]+1 & \text{【方案 A：删除】放弃} X[i] \\ D[i,j-1]+1 & \text{【方案 B：插入】补上} Y[j] \\ D[i-1,j-1]+\text{cost} & \text{【方案 C：替换/保留】字符对齐} \end{cases}$$

【重点解析】方案 C 的代价 (cost) 计算：

- 如果 $X[i] = Y[j]$，天生匹配！不需要操作，代价为 0。
- 如果 $X[i] \neq Y[j]$，需要替换。一次替换相当于“先删除后插入”两步操作，因此代价为 2。

以Levenshtein 距离变体计算为例，以上述算法过程如下；

```
function MIN-EDIT-DISTANCE(source, target) returns min-distance

  n ← LENGTH(source)
  m ← LENGTH(target)
  Create a distance matrix D[n+1,m+1]

  # Initialization: the zeroth row and column is the distance from the empty string
  D[0,0] = 0
  for each row i from 1 to n do
      D[i,0] ← D[i-1,0] + del-cost(source[i])
  for each column j from 1 to m do
      D[0,j] ← D[0,j-1] + ins-cost(target[j])

  # Recurrence relation:
  for each row i from 1 to n do
        for each column j from 1 to m do
          D[i,j] ← MIN( D[i−1,j] + del-cost(source[i]),
                        D[i−1,j−1] + sub-cost(source[i], target[j]),
                        D[i,j−1] + ins-cost(target[j]))
  # Termination
  return D[n,m]
```

Figure 2.17 The minimum edit distance algorithm, an example of the class of dynamic programming algorithms. The various costs can either be fixed (e.g., $\forall x, \text{ins-cost}(x) = 1$) or can be specific to the letter (to model the fact that some letters are more likely to be inserted than others). We assume that there is no cost for substituting a letter for itself (i.e., $\text{sub-cost}(x,x) = 0$).

计算单词intention和execution之间的Levenshtein 距离 (变体)；

Src\Tar	#	e	x	e	c	u	t	i	o	n
#	0	1	2	3	4	5	6	7	8	9
i	1	2	3	4	5	6	7	6	7	8
n	2	3	4	5	6	7	8	7	8	7
t	3	4	5	6	7	8	7	8	9	8
e	4	3	4	5	6	7	8	9	10	9
n	5	4	5	6	7	8	9	10	11	10
t	6	5	6	7	8	9	8	9	10	11
i	7	6	7	8	9	10	9	8	9	10
o	8	7	8	9	10	11	10	9	8	9
n	9	8	9	10	11	12	11	10	9	8

Figure 2.18 Computation of minimum edit distance between *intention* and *execution* with the algorithm of Fig. 2.17, using Levenshtein distance with cost of 1 for insertions or deletions, 2 for substitutions.

使用Levenshtein 距离 (变体)，找出单词intention和execution之间的最小编辑路径 (有多条)，黑色格子序列为其中一条路径；

	#	e	x	e	c	u	t	i	o	n
#	0	← 1	← 2	← 3	← 4	← 5	← 6	← 7	← 8	← 9
i	↑ **1**	↖←↑ 2	↖←↑ 3	↖←↑ 4	↖←↑ 5	↖←↑ 6	↖←↑ 7	↖ 6	← 7	← 8
n	↑ 2	↖←↑ **3**	↖←↑ 4	↖←↑ 5	↖←↑ 6	↖←↑ 7	↖←↑ 8	↑ 7	↖←↑ 8	↖ 7
t	↑ 3	↖←↑ 4	↖←↑ **5**	↖←↑ 6	↖←↑ 7	↖←↑ 8	↖ 7	←↑ 8	↖←↑ 9	↑ 8
e	↑ 4	↖ 3	← 4	↖← **5**	← **6**	← 7	←↑ 8	↖←↑ 9	↖←↑ 10	↑ 9
n	↑ 5	↑ 4	↖←↑ 5	↖←↑ 6	↖←↑ 7	↖←↑ **8**	↖←↑ 9	↖←↑ 10	↖←↑ 11	↖↑ 10
t	↑ 6	↑ 5	↖←↑ 6	↖←↑ 7	↖←↑ 8	↖←↑ 9	↖ **8**	← 9	← 10	←↑ 11
i	↑ 7	↑ 6	↖←↑ 7	↖←↑ 8	↖←↑ 9	↖←↑ 10	↑ 9	↖ **8**	← 9	← 10
o	↑ 8	↑ 7	↖←↑ 8	↖←↑ 9	↖←↑ 10	↖←↑ 11	↑ 10	↑ 9	↖ **8**	← 9
n	↑ 9	↑ 8	↖←↑ 9	↖←↑ 10	↖←↑ 11	↖←↑ 12	↑ 11	↑ 10	↑ 9	↖ **8**

Figure 2.19 When entering a value in each cell, we mark which of the three neighboring cells we came from with up to three arrows. After the table is full we compute an **alignment** (minimum edit path) by using a **backtrace**, starting at the **8** in the lower-right corner and following the arrows back. The sequence of bold cells represents one possible minimum cost alignment between the two strings, again using Levenshtein distance with cost of 1 for insertions or deletions, 2 for substitutions. Diagram design after Gusfield (1997).

题目1：编写Python代码实现下图中的最小编辑距离算法

```
function MIN-EDIT-DISTANCE(source, target) returns min-distance

  n ← LENGTH(source)
  m ← LENGTH(target)
  Create a distance matrix D[n+1,m+1]

  # Initialization: the zeroth row and column is the distance from the empty string
  D[0,0] = 0
  for each row i from 1 to n do
      D[i,0] ← D[i-1,0] + del-cost(source[i])
  for each column j from 1 to m do
      D[0,j] ← D[0,j-1] + ins-cost(target[j])

  # Recurrence relation:
  for each row i from 1 to n do
        for each column j from 1 to m do
          D[i,j] ← MIN( D[i−1,j] + del-cost(source[i]),
                        D[i−1,j−1] + sub-cost(source[i], target[j]),
                        D[i,j−1] + ins-cost(target[j]))
  # Termination
  return D[n,m]
```

Figure 2.17 The minimum edit distance algorithm, an example of the class of dynamic programming algorithms. The various costs can either be fixed (e.g., $\forall x, \text{ins-cost}(x) = 1$) or can be specific to the letter (to model the fact that some letters are more likely to be inserted than others). We assume that there is no cost for substituting a letter for itself (i.e., $\text{sub-cost}(x,x) = 0$).

题目2：主流大语言模型 Token 化机制对比与 API 成本预估 I

【背景说明】

大语言模型普遍采用子词分词算法，但各家底层的词表构建策略和切分粒度存在显著差异。这直接导致同一段文本在不同模型下的 Token 消耗量不同，进而深刻影响企业落地 AI 应用时的 **API 调用成本**和 **上下文窗口占用**。

【操作步骤与要求】

1. **语料准备（商务场景）：**
   截取一段真实的双语商务文本（约 200 字中文 + 200 词英文，如：财报摘要、产品新闻稿、电商客服对话等），作为标准测试语料。

2. **工具测量（多模型对比）：**
   使用至少 **3 种**不同大模型官方提供的 Tokenizer 工具（如 OpenAI Tiktokenizer、智谱 GLM Token 计算器等），对标准测试语料进行 Token 计数。

题目2：主流大语言模型 Token 化机制对比与 API 成本预估 II

3. **量化分析与业务思考：**
将实验结果整理成表格（包含：模型名称、纯中文 Token 数、纯英文 Token 数、总 Token 数），并回答以下问题：
   - **特征总结：**哪一款模型对中文的压缩效率最高（即生成 Token 最少）？这与该模型的训练背景有何关联？
   - **成本预估：**假设某金融企业每天需处理 10 万篇同等篇幅的文档。请查找其中两款模型最新的 API 计费标准，粗略计算并对比该企业一天的 API 输入成本差异。

【提交形式】

提交一页 A4 纸大小的 PDF 报告，需包含：测试语料原文、对比表格以及分析结论。

题目3：学习从语料库中挖掘计算语言学特征的顶刊论文

阅读下面这篇有关社交媒体语料库挖掘的顶刊论文，理解语料库中包含的多维度特征以及各维度特征随时间发生变化的规律：

Di Marco, Nicola, et al. "Patterns of linguistic simplification on social media platforms over time." Proceedings of the National Academy of Sciences 121.50 (2024): e2412105121. (点我下载)

拓展：借助大模型，理解论文中语言学特征的计算方法，使用Python实现这些计算方法，对论文提供的语料库（或者其他具有类似特征的语料库）计算语言学特征，加深理解。

THE END